МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовая работа
по курсу Операционные системы

Выполнил: А. А. Никитина
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов
Дата: 25.12.2025

Москва, 2025

1 Условие

Цель работы:

1. Приобретение практических навыков в использовании знаний, полученных в течении курса.
2. Проведение исследования в выбранной предметной области (сетевое взаимодействие, очереди сообщений).

Задание: Необходимо спроектировать и реализовать программный прототип в соответствии с вариантом. Произвести анализ и сделать вывод на основании данных, полученных при работе программного прототипа.

Вариант: 9

Консоль-серверная игра. Необходимо написать консоль-серверную игру. Необходимо написать 2 программы: сервер и клиент. Сначала запускается сервер, а далее клиенты соединяются с сервером. Сервер координирует клиентов между собой. При запуске клиента игрок может выбрать одно из следующих действий (возможно больше, если предусмотрено вариантом):

- Создать игру, введя ее имя
- Присоединиться к одной из существующих игр по имени игры

«Быки и коровы» (угадывать необходимо слова). Общение между сервером и клиентом необходимо организовать при помощи очередей сообщений (например, ZeroMQ). При создании каждой игры необходимо указывать количество игроков, которые будут участвовать. То есть угадывать могут несколько игроков. Если кто-то из игроков вышел из игры, то игра должна быть продолжена.

2 Метод решения

Для реализации поставленной задачи была выбрана архитектура «Клиент-Сервер». В качестве механизма межпроцессного взаимодействия (IPC) и сетевого взаимодействия используется библиотека **ZeroMQ**.

2.1 Обмен сообщениями

Используется паттерн обмена сообщениями `ROUTER-DEALER`:

- Сервер (ROUTER): Асинхронный сокет, способный принимать соединения от множества клиентов. Он автоматически добавляет идентификатор отправителя к входящим сообщениям, что позволяет маршрутизировать ответы конкретным клиентам.
- Клиент (DEALER): Сокет, который позволяет отправлять запросы и принимать ответы асинхронно. Это необходимо для того, чтобы клиент мог получать уведомления от сервера без блокировки ввода пользователя.

2.2 Протокол взаимодействия

Обмен данными происходит в текстовом формате. Сообщения имеют структуру: `COMMAND;ARG1;ARG2...`

Основные команды:

- `CREATE` — создание новой игровой сессии.
- `JOIN` — присоединение к существующей игре.
- `GUESS` — попытка угадать слово.
- `LEAVE` — выход из игры.

3 Описание программы

Проект разделен на несколько модулей:

3.1 Common – общая часть

- `os.h/cpp` – обертки над системными вызовами: реализован класс `Thread` (на базе `pthread_create`, `pthread_join`) для работы с потоками и функции обработки сигналов (`signal`).
- `bulls_and_cows.h/cpp` – логика игры, содержит алгоритм подсчета «быков» и «коров».
- `protocol.h` – константы протокола обмена.

3.2 Server – Серверная часть

- `server.cpp` – инициализация ZMQ контекста и сокета; основной цикл `Run` прослушивает входящие сообщения и передает их менеджеру игр. Используется мультиплексирование сообщений для определения адресата.
- `game.cpp (GameManager)` – хранит состояние всех активных игр и подключенных клиентов. Обрабатывает логику создания комнат и рассылку уведомлений (`Broadcast`) игрокам одной сессии.

3.3 Client – Клиентская часть

Клиент работает в два потока:

1. Главный поток: считывает команды пользователя с консоли (‘std::cin‘), формирует запросы и отправляет их на сервер.
2. Поток прослушивания: фоновый поток, который постоянно ожидает сообщения от сервера (результаты ходов, уведомления о подключении соперников) и выводит их на экран (доступ к консоли защищен мьютексом).

3.4 Точки входа – main.cpp

Два исполняемых файла, каждый из которых имеет собственную точку входа:
1. src/server/main.cpp – отвечает за инициализацию серверного приложения:

- устанавливает перехватчик системных сигналов. Это необходимо для корректного завершения работы сервера: при получении сигнала вызывается метод `Shutdown`, который уведомляет всех клиентов об остановке сервера перед выходом.

- создает экземпляр класса `server::Server` на порту 12345.
- запускает основной цикл обработки сообщений (`Run`).

2. src/client/main.cpp – отвечает за взаимодействие с пользователем и инициализацию клиента:

- Запрашивает имя пользователя.
- Инициализирует сокет ZeroMQ типа `DEALER` и устанавливает соединение с локальным хостом.
- Запускает отдельный поток (`receiver_thread`) для асинхронного приема сообщений от сервера.
- В главном потоке запускает цикл чтения команд из консоли (`std::cin`).
- Парсит введенные пользователем строки, преобразует их в протокольные сообщения (CREATE, JOIN, GUESS) и отправляет на сервер.
- При вводе команды `exit` корректно завершает поток приема и закрывает соединение.

3.5 Используемые функции для взаимодействия с ОС

- Работа с потоками:
  - `pthread_create()`: создание нового потока выполнения
  - `pthread_join()`: ожидание завершения ранее созданного потока.
- Синхронизация потоков:
  - `std::mutex`: реализации взаимного исключения (мьютекса) для защиты доступа к сокету и консоли из разных потоков.
  - `std::lock_guard`: автоматический захват и освобождение мьютекса, что предотвращает взаимоблокировки.
- Сетевое взаимодействие (библиотека ZeroMQ):
  - Функции библиотеки `ZeroMQ` (`zmq::socket_t`, `connect()`, `send()`, `recv()`): работа с очередями сообщений.
- Сигналы и управление временем:
  - `signal()`: устанавливает функции-обработчика для системных сигналов.
  - `usleep()`: приостанавление выполнения потока на указанное количество микросекунд.

4 Результаты работы

4.1 Сценарий тестирования

Для проверки работоспособности разработанного прототипа был проведен эксперимент с сервером и тремя клиентами (Nastya, Darya, Vasilisa). Сценарий включал следующие этапы:

1. Запуск сервера.
2. Подключение клиента Nastya, создание игры `test_game` на 3 игроков.
3. Подключение клиентов Darya и Vasilisa, присоединение к игре.
4. Игровой процесс: несколько игроков поочередно отправляют варианты слова.
5. Тестирование ситуации: клиент Nastya выходит из игры до ее завершения.
6. Завершение игры: клиент Vasilisa угадывает слово.
7. Тестирование корректного завершения: сервер останавливается по сигналу `SIGINT` (Ctrl+C), уведомляя оставшихся клиентов.

4.2 Полученные данные

В ходе работы программного прототипа были получены следующие данные.

4.2.1 Лог сервера

На сервере видно последовательную обработку запросов от всех клиентов, корректное завершение сессии при выходе игрока и победе, а также перехват сигнала `SIGINT` для graceful shutdown.

```
./server
Server started on port 12345
[REQ] From 'Nastya': CREATE...
[CREATE] Game 'test_game' created by 'Nastya'.
[REQ] From 'Darya': JOIN...
[JOIN] Player 'Darya' joined game 'test_game'.
[REQ] From 'Vasilisa': JOIN...
[JOIN] Player 'Vasilisa' joined game 'test_game'.
[START] Game 'test_game' started!
[REQ] From 'Nastya': GUESS...
[GUESS] Game 'test_game', Player 'Nastya', Word 'ADVENTURE'
[REQ] From 'Darya': GUESS...
[GUESS] Game 'test_game', Player 'Darya', Word 'CHALLENGE'
[REQ] From 'Nastya': LEAVE...
[DISCONNECT] Player Nastya left game test_game
[REQ] From 'Darya': GUESS...
[GUESS] Game 'test_game', Player 'Darya', Word 'SEMAPHORE'
[REQ] From 'Vasilisa': GUESS...
[GUESS] Game 'test_game', Player 'Vasilisa', Word 'SCHEDULER'
[WIN] Player 'Vasilisa' won game 'test_game'!
^C
```

```
[SIGINT] Termination signal received.
[SYSTEM] Shutting down. Notifying all clients...
```

4.2.2 Лог клиента-победителя Vasilisa

Лог клиента наглядно демонстрирует получение сообщений о ходе игры, реакции на попытки других игроков и корректное завершение при остановке сервера.

```
./client
Enter your name: Vasilisa
...
> join test_game
[OK] Joined test_game
[GAME] Player 'Vasilisa' joined (3/3)
[GAME] Game started! Word len: 9
[GAME] 'Nastya' guess 'ADVENTURE': 1B 4C
[GAME] 'Darya' guess 'CHALLENGE': 0B 5C
[GAME] Player 'Nastya' has left. Slot opened!
[GAME] 'Darya' guess 'SEMAPHORE': 1B 4C
> guess test_game SCHEDULER
[GAME] Player 'Vasilisa' WON! Word: SCHEDULER
[GAME]
[SYSTEM] Server stopped. Exiting...
```

4.3 Анализ работы прототипа

На основании полученных данных проведен анализ ключевых характеристик системы:

1. Синхронизация состояний и широковещательная рассылка
Логи клиентов (например, Vasilisa) показывают, что все уведомления от сервера — о присоединении игроков, о начале игры, о ходах соперников, о выходе игрока — доставляются и отображаются корректно. Это подтверждает, что механизм широковещательной рассылки (`Broadcast`) на сервере работает правильно.

2. Устойчивость к потере клиентов
Когда игрок Nastya выходит из игры (`exit`), сервер корректно обрабатывает это событие (`[DISCONNECT]`), и оставшиеся игроки (Darya и Vasilisa) получают уведомление `Player 'Nastya' has left`. Игра продолжается без сбоев.

3. Корректность игровой логики
Результаты подсчета "быков"и "коров"для слов `ADVENTURE`, `CHALLENGE` и `SEMAPHORE` соответствуют загаданному слову `SCHEDULER`. Лог клиента Vasilisa наглядно демонстрирует процесс отгадывания: неверная попытка (`CHALLENGE`) приводит к результату `0B 5C`, а верная — к победе.

4. Механизм корректного завершения
После остановки сервера по сигналу `SIGINT`, все активные клиенты получают системное сообщение `[SYSTEM] Server stopped. Exiting...` и завершают свою работу. Это показывает, что обработчик сигналов на сервере и логика уведомления клиентов реализованы успешно, предотвращая "зависание"клиентов при падении сервера.

5 Выводы

В рамках курсового проекта была успешно спроектирована, реализована и протестирована клиент-серверная система для многопользовательской игры «Быки и коровы». Раз-

работанный программный прототип демонстрирует практическое применение ключевых концепций операционных систем, таких как межпроцессное взаимодействие, многопоточность и обработка сигналов.

Основой для построения эффективной и неблокирующей системы послужил выбор библиотеки ZeroMQ и асинхронного паттерна обмена сообщениями `ROUTER-DEALER`. Такой подход позволил серверу инициировать отправку данных клиентам в любой момент времени. Это обеспечило мгновенную доставку уведомлений о ходе игры всем участникам, что являетс важным для интерактивных приложений.

На клиентской стороне была реализована многопоточная модель, разделяющая логику пользовательского интерфейса и сетевого взаимодействия. Фоновый поток, отвечающий за прием сообщений, позволил избежать блокировки основного потока и обеспечить высокую отзывчивость приложения. Для предотвращения состояния гонки при доступе к общим ресурсам, таким как консоль и сокет, были использованы примитивы синхронизации `std::mutex`. Кроме того, система показала высокую устойчивость к нештатным ситуациям: реализован механизм graceful shutdown по сигналу `SIGINT` и корректная обработка отключения игроков, что позволяет продолжать игровую сессию без сбоев.

Таким образом, цели курсового проекта были полностью достигнуты. Разработанный программный прототип является рабочим решением, которое демонстрирует глубокое понимание рассмотренных в курсе тем и способность применять теоретические знания для решения прикладных задач.

6 Исходный код программы

```
#pragma once

#include <string>
#include <utility>

namespace core {

    class BullsAndCowsGame {
    public:
        explicit BullsAndCowsGame(std::string secret_word);

        std::pair<int, int> CheckGuess(const std::string& guess) const;

        bool IsWordGuessed(const std::string& guess) const;

        int GetWordLength() const;

    private:
        std::string _secret_word;
    };

}
```

Листинг 1: include/common/bulls_and_cows.h

```
#pragma once

#include <stdexcept>
#include <string>

namespace except {

    class OsExcept : public std::runtime_error {
    public:
        explicit OsExcept(const std::string& message) : std::
    runtime_error(message) {}
    };

    class ThreadCreateExcept : public OsExcept {
    public:
        explicit ThreadCreateExcept(const std::string& message) :
    OsExcept(message) {}
    };

    class ThreadJoinExcept : public OsExcept {
    public:
        explicit ThreadJoinExcept(const std::string& message) :
    OsExcept(message) {}
    };

    class GameException : public std::runtime_error {
    public:
```

```
        explicit GameException(const std::string& message) : std::runtime_error(message) {}
    };

}
```

Листинг 2: include/common/exceptions.h

```
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "bulls_and_cows.h"

namespace core {

    struct Player {
        std::string name;
        std::string cur_game;
    };

    struct GameSession {
        std::string name;
        std::unique_ptr<BullsAndCowsGame> game;
        size_t max_players;
        std::vector<std::string> player_identities;
        bool is_started = false;
        std::string winner;
    };

}
```

Листинг 3: include/common/models.h

```
#pragma once

#include <sstream>
#include <string>
#include <vector>


namespace protocol {

    const char DELIMITER = ';';
    const std::string REQ_CREATE = "CREATE";
    const std::string REQ_JOIN = "JOIN";
    const std::string REQ_GUESS = "GUESS";
    const std::string REQ_LEAVE = "LEAVE";
    const std::string REP_SUCCESS = "SUCCESS";
    const std::string REP_ERROR = "ERROR";
    const std::string REP_NOTIFY = "NOTIFY";
    const std::string REP_SERVER_STOP = "SERVER_STOP";
```

```

}

namespace utils {

    inline std::vector<std::string> Split(const std::string& s, char delimiter) {
        std::vector<std::string> tokens;
        std::string token;
        std::istringstream tokenStream(s);
        while (std::getline(tokenStream, token, delimiter)) {
            tokens.push_back(token);
        }
        return tokens;
    }

}
```

Листинг 4: include/common/protocol.h

```
#pragma once

#include <pthread.h>
#include <stdexcept>
#include <string>

namespace os {

    using ThreadFunc = void* (*)(void*);
    struct ThreadInfo;
    using ThreadHandle = ThreadInfo*;
    using SignalHandler = void (*)(int);

    class Thread {
        private:
        ThreadFunc func_;
        ThreadHandle handle_;

        public:
        Thread(ThreadFunc func);

        Thread(const Thread&) = delete;

        Thread& operator=(const Thread&) = delete;

        Thread(Thread&& other);

        Thread& operator=(Thread&& other);

        void Run (void* thread_data);

        void Join();

        void Swap(Thread& other);
```

```

        ~Thread() noexcept;

    };

    void Sleep(int milliseconds);

    void SetInterruptHandler(SignalHandler handler);

}
```

Листинг 5: include/common/os.h

```
#pragma once

#include <map>
#include <string>
#include <vector>

#include "models.h"

namespace server {

    class Server;

    class GameManager {
    public:
        explicit GameManager(Server& server);

        void ClientDisconnected(const std::string& client_identity);

        void MessageReceived(const std::string& client_identity, const std::string& message);

        void Shutdown();

    private:
        void CreateGame(const std::string& identity, const std::vector<std::string>& args);

        void JoinGame(const std::string& identity, const std::vector<std::string>& args);

        void Guess(const std::string& identity, const std::vector<std::string>& args);

        std::string RandomWord() const;

        std::map<std::string, core::Player> _clients;
        std::map<std::string, core::GameSession> _active;
        Server& _server;
        std::vector<std::string> _words;
    };
}
```

Листинг 6: include/server/game.h

```
#pragma once

#include <map>
#include <memory>
#include <string>
#include <vector>
#include <zmq.hpp>

#include "game.h"

namespace server {

    class Server {
    public:
        explicit Server(int port);

        ~Server();

        void Run();

        void SendMessage(const std::string& client_identity, const std
    ::string& message);

        void Broadcast(const std::vector<std::string>&
    client_identities, const std::string& message);

        void Shutdown();

    private:
        int _port;
        std::unique_ptr<GameManager> _game_manager;
        zmq::context_t _context;
        zmq::socket_t _socket;
    };

}
```

Листинг 7: include/server/server.h

```
#include <atomic>
#include <ctime>
#include <iostream>
#include <mutex>
#include <string>
#include <vector>
#include <zmq.hpp>

#include "exceptions.h"
#include "os.h"
#include "protocol.h"

```

```
namespace client {

    std::mutex g_socket_mutex;
    std::mutex g_console_mutex;
    std::atomic<bool> g_running(true);

    struct ThreadArgs {
        zmq::socket_t* socket;
    };

    void* receive_messages(void* arg) {
        auto* args = static_cast<ThreadArgs*>(arg);

        while (g_running) {
            zmq::message_t msg_delim, msg_payload;
            bool received = false;

            {
                std::lock_guard<std::mutex> lock(g_socket_mutex);
                try {
                    auto res = args->socket->recv(msg_delim, zmq::recv_flags::dontwait);
                    if (res) {
                        if (msg_delim.size() == 0) {
                            auto res_payload = args->socket->recv(msg_payload, zmq::recv_flags::none);
                            if (res_payload) received = true;
                        } else {
                            msg_payload = std::move(msg_delim);
                            received = true;
                        }
                    }
                } catch (zmq::error_t&) {
                }
            }

            if (received) {
                std::string text = msg_payload.to_string();
                auto parts = utils::Split(text, protocol::DELIMITER);

                if (!parts.empty() && parts[0] == protocol::REP_SERVER_STOP) {
                    {
                        std::lock_guard<std::mutex> console_lock(g_console_mutex);
                        std::cout << "\r" << std::string(50, ' ') << "\r";
                        std::cout << "[SYSTEM] Server stopped. Exiting..." << std::endl;
                    }
                    g_running = false;
                    exit(0);
                }
```

```

                std::string display_text = text;
                std::string prefix = "[GAME]";

                if (!parts.empty()) {
                    if (parts[0] == protocol::REP_ERROR) prefix = "[ERROR]";
                    else if (parts[0] == protocol::REP_SUCCESS) prefix = "[OK]";

                    if (parts.size() > 1 && (parts[0] == protocol::REP_SUCCESS || parts[0] == protocol::REP_ERROR || parts[0] == protocol::REP_NOTIFY)) {
                        display_text = parts[1];
                    }
                }

                {
                    std::lock_guard<std::mutex> console_lock(g_console_mutex);
                    std::cout << "\r" << std::string(50, ' ') << "\r";
                    std::cout << prefix << " " << display_text << std::endl;
                    std::cout << "> " << std::flush;
                }
            } else {
                os::Sleep(50);
            }
        }
        return nullptr;
    }
}

int main() {
    std::string player_name;
    std::cout << "Enter your name: ";
    std::getline(std::cin, player_name);

    if (player_name.empty()) player_name = "Player" + std::to_string(std::time(nullptr));

    zmq::context_t context(1);
    zmq::socket_t client_socket(context, zmq::socket_type::dealer);

    client_socket.set(zmq::sockopt::routing_id, player_name);

    try {
        client_socket.connect("tcp://127.0.0.1:12345");
    } catch (const zmq::error_t& e) {
        std::cerr << "Failed to connect: " << e.what() << std::endl;
        return 1;
    }

```

```
    std::cout << "Connected! Commands:\n"
              << "  create <game> <players_count>\n"
              << "  join <game>\n"
              << "  guess <game> <WORD>\n"
              << "  exit\n" << std::endl;

    client::ThreadArgs args = { &client_socket };

    try {
        os::Thread receiver_thread(client::receive_messages);
        receiver_thread.Run(&args);

        std::string line;
        std::cout << "> " << std::flush;

        while (std::getline(std::cin, line)) {
            if (line == "exit") break;
            if (line.empty()) {
                std::cout << "> " << std::flush;
                continue;
            }

            auto parts = utils::Split(line, ' ');
            std::string request_str;
            const std::string& cmd = parts[0];

            if (cmd == "create" && parts.size() == 3) {
                request_str = protocol::REQ_CREATE + protocol::DELIMITER + parts[1] + protocol::DELIMITER + parts[2] + protocol::DELIMITER + player_name;
            } else if (cmd == "join" && parts.size() == 2) {
                request_str = protocol::REQ_JOIN + protocol::DELIMITER + parts[1] + protocol::DELIMITER + player_name;
            } else if (cmd == "guess" && parts.size() == 3) {
                request_str = protocol::REQ_GUESS + protocol::DELIMITER + parts[1] + protocol::DELIMITER + player_name + protocol::DELIMITER + parts[2];
            } else {
                std::lock_guard<std::mutex> lock(client::g_console_mutex);
                std::cout << "Unknown command.\n> " << std::flush;
                continue;
            }

            {
                std::lock_guard<std::mutex> lock(client::g_socket_mutex);
                client_socket.send(zmq::buffer(""), zmq::send_flags::sndmore);
                client_socket.send(zmq::buffer(request_str), zmq::send_flags::none);
            }

```

```
            std::lock_guard<std::mutex> lock(client::g_console_mutex);
            std::cout << "> " << std::flush;
        }

        {
             std::lock_guard<std::mutex> lock(client::g_socket_mutex);
             std::string leave_msg = protocol::REQ_LEAVE + protocol::
    DELIMITER + "LEAVE";
             client_socket.send(zmq::buffer(""), zmq::send_flags::
    sndmore);
             client_socket.send(zmq::buffer(leave_msg), zmq::send_flags
    ::dontwait);
        }

        client::g_running = false;
        receiver_thread.Join();

    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    return 0;
}
```

Листинг 8: src/client/main.cpp

```
#include <algorithm>

#include "bulls_and_cows.h"

namespace core {

    BullsAndCowsGame::BullsAndCowsGame(std::string secret_word)
        : _secret_word(std::move(secret_word)) {}

    bool BullsAndCowsGame::IsWordGuessed(const std::string& guess)
    const {
        return guess == _secret_word;
    }

    int BullsAndCowsGame::GetWordLength() const {
        return _secret_word.length();
    }

    std::pair<int, int> BullsAndCowsGame::CheckGuess(const std::string&
    guess) const {
        if (guess.length() != _secret_word.length()) {
            return {0, 0};
        }
        int bulls = 0;
        int cows = 0;
        int secret_word_freq[256] = {0};
        int guess_freq[256] = {0};
```

```

        for (size_t i = 0; i < _secret_word.length(); ++i) {
            if (_secret_word[i] == guess[i]) {
                bulls++;
            } else {
                secret_word_freq[static_cast<unsigned char>(_secret_word[i])]++;
                guess_freq[static_cast<unsigned char>(guess[i])]++;
            }
        }
        for (int i = 0; i < 256; ++i) {
            cows += std::min(secret_word_freq[i], guess_freq[i]);
        }
        return {bulls, cows};
    }

}
```

Листинг 9: src/common/bulls_and_cows.cpp

```
#include <algorithm>
#include <csignal>
#include <unistd.h>

#include "exceptions.h"
#include "os.h"

namespace os {

    struct ThreadInfo {
        pthread_t thread;
    };

    Thread::Thread(ThreadFunc func): func_(func) {
        handle_ = new ThreadInfo();
    }

    Thread::Thread(Thread&& other): func_(other.func_), handle_(other.handle_) {
        other.func_ = nullptr;
        other.handle_ = nullptr;
    }

    Thread& Thread::operator=(Thread&& other) {
        if (this == &other) {
            return *this;
        }
        Thread temp = std::move(other);
        this->Swap(temp);
        return *this;
    }

    void Thread::Run (void* thread_data) {
        int res = pthread_create(&(handle_->thread), NULL, func_,
```

```
    thread_data);
        if (res != 0) {
            throw except::ThreadCreateExcept("Failed to create a thread.");
        }
    }

    void Thread::Join() {
        int res = pthread_join(handle_->thread, NULL);
        if (res != 0) {
            throw except::ThreadJoinExcept("Failed to join a thread.");
        }
    }

    void Thread::Swap(Thread& other) {
        std::swap(func_, other.func_);
        std::swap(handle_, other.handle_);
    }

    Thread::~Thread() noexcept {
        delete handle_;
    }

    void Sleep(int milliseconds) {
        usleep(milliseconds * 1000);
    }

    void SetInterruptHandler(SignalHandler handler) {
        std::signal(SIGINT, handler);
    }

}
```

Листинг 10: src/common/os.cpp

```
#include <algorithm>
#include <iostream>
#include <random>

#include "exceptions.h"
#include "protocol.h"
#include "game.h"
#include "server.h"

namespace server {

    GameManager::GameManager(Server& server) : _server(server) {
        _words = {
            "PROCESS", "THREAD", "MUTEX", "SEMAPHORE", "KERNEL",
            "SOCKET", "MEMORY", "PAGING", "SCHEDULER", "DEADLOCK"
        };
    }

    void GameManager::MessageReceived(const std::string&
```

```
client_identity, const std::string& message) {
    if (_clients.find(client_identity) == _clients.end()) {
        _clients[client_identity] = core::Player{"Unknown", ""};
    }

    auto parts = utils::Split(message, protocol::DELIMITER);
    if (parts.empty()) return;

    const std::string& command = parts[0];

    std::cout << "[REQ] From '" << client_identity << "': " << command << std::endl;

    try {
        if (command == protocol::REQ_CREATE) {
            CreateGame(client_identity, parts);
        } else if (command == protocol::REQ_JOIN) {
            JoinGame(client_identity, parts);
        } else if (command == protocol::REQ_GUESS) {
            Guess(client_identity, parts);
        } else if (command == protocol::REQ_LEAVE) {
            ClientDisconnected(client_identity);
        }
    } catch (const except::GameException& e) {
        std::cout << "[FAIL] User '" << client_identity << "': " << e.what() << std::endl;
        _server.SendMessage(client_identity, protocol::REP_ERROR + protocol::DELIMITER + e.what());
    } catch (const std::exception& e) {
        std::cerr << "[CRITICAL] " << e.what() << std::endl;
        _server.SendMessage(client_identity, protocol::REP_ERROR + protocol::DELIMITER + "Internal server error");
    }
}

void GameManager::Shutdown() {
    std::cout << "[SYSTEM] Shutting down. Notifying all clients..." << std::endl;
    for (const auto& [identity, player] : _clients) {
        try {
            _server.SendMessage(identity, protocol::REP_SERVER_STOP);
        } catch (...) {

        }
    }
}

void GameManager::ClientDisconnected(const std::string& client_identity) {
    if (!_clients.count(client_identity)) return;

    core::Player& player = _clients.at(client_identity);
```

```
        std::string game_name = player.cur_game;
        std::string player_name = player.name;

        if (!game_name.empty() && _active.count(game_name)) {
            auto& game = _active.at(game_name);
            auto& ids = game.player_identities;

            ids.erase(std::remove(ids.begin(), ids.end(), client_identity), ids.end());

            std::cout << "[DISCONNECT] Player " << player_name << " left game " << game_name << std::endl;

            if (ids.empty()) {
                _active.erase(game_name);
                std::cout << "[GAME DESTROY] Game " << game_name << " is empty and deleted." << std::endl;
            } else {
                std::string msg = "Player '" + player_name + "' has left. Slot opened!";
                _server.Broadcast(game.player_identities, protocol::REP_NOTIFY + protocol::DELIMITER + msg);
            }
        } else {
            std::cout << "[DISCONNECT] User " << client_identity << " disconnected (was not in game)." << std::endl;
        }

        player.cur_game = "";
        _clients.erase(client_identity);
    }

    void GameManager::CreateGame(const std::string& identity, const std::vector<std::string>& args) {
        if (!_clients[identity].cur_game.empty()) {
            throw except::GameException("You are already in game '" + _clients[identity].cur_game + "'. Type 'exit' first or restart client.");
        }

        if (args.size() != 4) {
            throw except::GameException("Usage: create <name> <count>");
        }
        const std::string& game_name = args[1];

        size_t player_count;
        try {
            player_count = std::stoul(args[2]);
        } catch (...) {
            throw except::GameException("Invalid number of players.");
        }
        const std::string& player_name = args[3];
```

```
        if (_active.count(game_name)) {
            throw except::GameException("Game already exists.");
        }

        _clients[identity].name = player_name;
        _clients[identity].cur_game = game_name;

        core::GameSession new_game;
        new_game.name = game_name;
        new_game.game = std::make_unique<core::BullsAndCowsGame>(RandomWord());
        new_game.max_players = player_count;
        new_game.player_identities.push_back(identity);

        _active[game_name] = std::move(new_game);

        std::cout << "[CREATE] Game '" << game_name << "' created by '" << player_name << "'." << std::endl;

        _server.SendMessage(identity, protocol::REP_SUCCESS + protocol::DELIMITER + "Game created. Waiting for opponents...");

        if (player_count == 1) {
            _active[game_name].is_started = true;
            std::string info = "Started! Word len: " + std::to_string(_active[game_name].game->GetWordLength());
            _server.SendMessage(identity, protocol::REP_NOTIFY + protocol::DELIMITER + info);
            std::cout << "[START] Game '" << game_name << "' auto-started (1 player)." << std::endl;
        }
    }

    void GameManager::JoinGame(const std::string& identity, const std::vector<std::string>& args) {
        if (!_clients[identity].cur_game.empty()) {
            throw except::GameException("You are already in game '" + _clients[identity].cur_game + "'.");
        }

        if (args.size() != 3) {
            throw except::GameException("Usage: join <game_name>");
        }
        const std::string& game_name = args[1];
        const std::string& player_name = args[2];

        if (!_active.count(game_name)) {
            throw except::GameException("Game not found.");
        }

        auto& game = _active.at(game_name);

```

```
        for (const auto& pid : game.player_identities) {
            if (_clients.at(pid).name == player_name) {
                throw except::GameException("Name already taken in this game.");
            }
        }

        if (game.player_identities.size() >= game.max_players) {
            throw except::GameException("Game is full.");
        }

        _clients[identity].name = player_name;
        _clients[identity].cur_game = game_name;
        game.player_identities.push_back(identity);

        std::cout << "[JOIN] Player '" << player_name << "' joined game '" << game_name << "'." << std::endl;

        _server.SendMessage(identity, protocol::REP_SUCCESS + protocol::DELIMITER + "Joined " + game_name);

        std::string join_msg = "Player '" + player_name + "' joined (" +
                                std::to_string(game.player_identities.size()) + "/" +
                                std::to_string(game.max_players) + ")";
        _server.Broadcast(game.player_identities, protocol::REP_NOTIFY + protocol::DELIMITER + join_msg);

        if (game.player_identities.size() == game.max_players && !game.is_started) {
            game.is_started = true;
            std::string start_msg = "Game started! Word len: " + std::to_string(game.game->GetWordLength());
            _server.Broadcast(game.player_identities, protocol::REP_NOTIFY + protocol::DELIMITER + start_msg);
            std::cout << "[START] Game '" << game_name << "' started!" << std::endl;
        } else if (game.is_started) {
             std::string info_msg = "Game is already in progress! Word len: " + std::to_string(game.game->GetWordLength());
             _server.SendMessage(identity, protocol::REP_NOTIFY + protocol::DELIMITER + info_msg);
             std::cout << "[INFO] Sent game info to new player '" << player_name << "'." << std::endl;
        }
    }

    void GameManager::Guess(const std::string& identity, const std::vector<std::string>& args) {
        if (args.size() != 4) {
             throw except::GameException("Usage: guess <game> <word>");
        }
```

```
        const std::string& game_name = args[1];
        const std::string& word = args[3];
        const std::string& player_name = _clients[identity].name;

        if (_clients[identity].cur_game != game_name) {
             throw except::GameException("You are not in game '" + game_name + "'");
        }

        if (!_active.count(game_name) || !_active.at(game_name).is_started) {
            throw except::GameException("Game not active.");
        }

        auto& game = _active.at(game_name);
        if (!game.winner.empty()) {
            throw except::GameException("Game is over. Winner: " + game.winner);
        }

        std::cout << "[GUESS] Game '" << game_name << "', Player '" << player_name << "', Word '" << word << "'" << std::endl;

        if (game.game->IsWordGuessed(word)) {
            game.winner = player_name;
            std::string win_msg = "Player '" + player_name + "' WON! Word: " + word;
            _server.Broadcast(game.player_identities, protocol::REP_NOTIFY + protocol::DELIMITER + win_msg);
            std::cout << "[WIN] Player '" << player_name << "' won game '" << game_name << "'!" << std::endl;
        } else {
            auto [bulls, cows] = game.game->CheckGuess(word);
            std::string res = "'" + player_name + "' guess '" + word + "': " +
                                std::to_string(bulls) + "B " + std::to_string(cows) + "C";
            _server.Broadcast(game.player_identities, protocol::REP_NOTIFY + protocol::DELIMITER + res);
        }
    }

    std::string GameManager::RandomWord() const {
        if (_words.empty()) return "LINUX";
        static std::mt19937 gen(std::random_device{}());
        std::uniform_int_distribution<> distrib(0, _words.size() - 1);
        return _words[distrib(gen)];
    }

}
```

Листинг 11: src/server/game.cpp

```
#include <iostream>
```

```

#include "server.h"

namespace server {

    Server::Server(int port)
        : _port(port),
        _context(1),
        _socket(_context, zmq::socket_type::router) {
        std::string addr = "tcp://*:" + std::to_string(_port);
        _socket.bind(addr);
        _game_manager = std::make_unique<GameManager>(*this);
        std::cout << "Server started on port " << _port << std::endl;
    }

    Server::~Server() = default;

    void Server::Run() {
        while (true) {
            std::vector<zmq::message_t> recv_parts;
            zmq::message_t part;

            try {
                auto result = _socket.recv(part);
                if (!result) continue;
                recv_parts.push_back(std::move(part));

                while (_socket.get(zmq::sockopt::rcvmore)) {
                    zmq::message_t next_part;
                    auto res = _socket.recv(next_part);
                    if (!res) break;
                    recv_parts.push_back(std::move(next_part));
                }

                if (recv_parts.size() < 3) continue;

                std::string client_identity(static_cast<char*>(recv_parts[0].data()), recv_parts[0].size());
                std::string payload(static_cast<char*>(recv_parts[2].data()), recv_parts[2].size());

                _game_manager->MessageReceived(client_identity, payload);

            } catch (const zmq::error_t& e) {
                std::cerr << "ZMQ Error in Run: " << e.what() << std::endl;
            }
        }
    }

    void Server::SendMessage(const std::string& client_identity, const std::string& message) {
```

```
        try {
            _socket.send(zmq::buffer(client_identity), zmq::send_flags
    ::sndmore);
            _socket.send(zmq::buffer(""), zmq::send_flags::sndmore);
            _socket.send(zmq::buffer(message), zmq::send_flags::none);
        } catch (const zmq::error_t& e) {
            std::cerr << "Failed to send message to " <<
    client_identity << ": " << e.what() << std::endl;
        }
    }

    void Server::Broadcast(const std::vector<std::string>&
    client_identities, const std::string& message) {
        for (const auto& identity : client_identities) {
            SendMessage(identity, message);
        }
    }

    void Server::Shutdown() {
        if (_game_manager) {
            _game_manager->Shutdown();
        }
    }

}
```

Листинг 12: src/server/server.cpp

```
#include <csignal>
#include <iostream>

#include "os.h"
#include "server.h"

server::Server* g_server_ptr = nullptr;

void signal_handler(int) {
    if (g_server_ptr) {
        std::cout << "\n[SIGINT] Termination signal received." << std::
    endl;
        g_server_ptr->Shutdown();
    }
    exit(0);
}

int main() {
    os::SetInterruptHandler(signal_handler);

    try {
        server::Server server(12345);
        g_server_ptr = &server;
        server.Run();

    } catch (const std::exception& e) {
```

```
        std::cerr << "Fatal server error: " << e.what() << std::endl;
        return 1;
    }
    return 0;
}
```

Листинг 13: src/server/main.cpp

```
cmake_minimum_required(VERSION 3.10)
project(BullsAndCows)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -pthread")

find_package(PkgConfig REQUIRED)
pkg_check_modules(ZMQ REQUIRED libzmq)
include_directories(${ZMQ_INCLUDE_DIRS})

include_directories(${CMAKE_SOURCE_DIR}/include/common)
include_directories(${CMAKE_SOURCE_DIR}/include/server)

add_library(common
    src/common/bulls_and_cows.cpp
    src/common/os.cpp
)

target_link_libraries(common PRIVATE ${ZMQ_LIBRARIES})

add_executable(server
    src/server/main.cpp
    src/server/game.cpp
    src/server/server.cpp
)

target_link_libraries(server PRIVATE common ${ZMQ_LIBRARIES})

add_executable(client
    src/client/main.cpp
)

target_link_libraries(client PRIVATE common ${ZMQ_LIBRARIES})
```

Листинг 14: CMakeLists.txt

Вывод утилиты strace

```
27244 execve("./client", ["./client"], 0x7ffe5e89bc28 /* 27 vars */) =
    0
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\240\233\1\0\0\0\0\0"...,
    832) = 832
```

```
27244 read(3, "\177ELF
    \2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P
    \237\2\0\0\0\0\0"..., 832) = 832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\340L
    \0\0\0\0\0\0"..., 832) = 832
27244 read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0
    \255\0\0\0\0\0\0"..., 832) = 832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 read(3, "\177ELF
    \2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) =
    832
27244 write(1, "Enter your name: ", 17) = 17
27244 read(0, "Nastya\n", 1024)         = 7
27244 read(3, "0-11\n", 1024)           = 5
27244 read(3, "# /etc/nsswitch.conf\n#\n# Example"..., 4096) = 510
27244 read(3, "", 4096)                 = 0
```

```
27244 read(3, "# Internet (IP) protocols\n#\n# Up"..., 4096) = 2932
27244 read(3, "", 4096)                 = 0
27244 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
   CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
   CLONE_CHILD_CLEARTID, child_tid=0x7fb151ce7910, parent_tid=0
   x7fb151ce7910, exit_signal=0, stack=0x7fb1514e7000, stack_size=0
   x7ffc80, tls=0x7fb151ce7640} => {parent_tid=[27245]}, 88) = 27245
27244 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
   CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
   CLONE_CHILD_CLEARTID, child_tid=0x7fb1514e6910, parent_tid=0
   x7fb1514e6910, exit_signal=0, stack=0x7fb150ce6000, stack_size=0
   x7ffc80, tls=0x7fb1514e6640} => {parent_tid=[27246]}, 88) = 27246
27244 write(6, "\1\0\0\0\0\0\0\0", 8)   = 8
27244 write(8, "\1\0\0\0\0\0\0\0", 8)   = 8
27244 write(1, "Connected! Commands:\n", 21) = 21
27244 write(1, "  create <game> <players_count>\n", 32 <unfinished ...>
27246 read(6,  <unfinished ...>
27244 <... write resumed>)              = 32
27246 <... read resumed>"\1\0\0\0\0\0\0\0", 8) = 8
27244 write(1, "  join <game>\n", 14)   = 14
27244 write(1, "  guess <game> <WORD>\n", 22) = 22
27244 write(1, "  exit\n", 7)           = 7
27244 write(1, "\n", 1)                 = 1
27246 socket(AF_INET, SOCK_STREAM|SOCK_CLOEXEC, IPPROTO_TCP <unfinished
    ...>
27244 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
   CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
   CLONE_CHILD_CLEARTID, child_tid=0x7fb150ce5910, parent_tid=0
   x7fb150ce5910, exit_signal=0, stack=0x7fb1504e5000, stack_size=0
   x7ffc80, tls=0x7fb150ce5640} => {parent_tid=[27247]}, 88) = 27247
27244 write(1, "> ", 2)                 = 2
27244 read(0,  <unfinished ...>
27247 read(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27246 <... socket resumed>)             = 9
27246 connect(9, {sa_family=AF_INET, sin_port=htons(12345), sin_addr=
   inet_addr("127.0.0.1")}, 16) = -1 EINPROGRESS (Operation now in
   progress)
27246 getsockopt(9, SOL_SOCKET, SO_ERROR, [0], [4]) = 0
27246 setsockopt(9, SOL_TCP, TCP_NODELAY, [1], 4) = 0
27246 getsockname(9, {sa_family=AF_INET, sin_port=htons(54902),
   sin_addr=inet_addr("127.0.0.1")}, [128 => 16]) = 0
27246 getpeername(9, {sa_family=AF_INET, sin_port=htons(12345),
   sin_addr=inet_addr("127.0.0.1")}, [128 => 16]) = 0
27246 write(6, "\1\0\0\0\0\0\0\0", 8)   = 8
27246 read(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27246 recvfrom(9, "\377\0\0\0\0\0\0\0\1\177", 12, 0, NULL, NULL) = 10
27246 recvfrom(9, 0x7fb14c001bf2, 2, 0, NULL, NULL) = -1 EAGAIN (
   Resource temporarily unavailable)
27246 sendto(9, "\377\0\0\0\0\0\0\0\7\177\3", 11, 0, NULL, 0) = 11
27246 recvfrom(9, "\3\1", 2, 0, NULL, NULL) = 2
27246 recvfrom(9, "NULL
   \0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 52, 0,
    NULL, NULL) = 52
```

```
27246 recvfrom(9, 0x7fb14c003d18, 8192, 0, NULL, NULL) = -1 EAGAIN (Resource temporarily unavailable)
27246 sendto(9, "\1NULL\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 53, 0, NULL, 0) = 53
27246 sendto(9, "\4/\5READY\vSocket-Type\0\0\0\6DEALER\10I"..., 49, 0, NULL, 0) = 49
27246 recvfrom(9, "\4)\5READY\vSocket-Type\0\0\0\6ROUTER\10I"..., 8192, 0, NULL, NULL) = 43
27244 <... read resumed>"create game 2\n", 1024) = 14
27244 write(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27244 write(1, "> ", 2)                  = 2
27244 read(0,  <unfinished ...>
27246 read(6, "\1\0\0\0\0\0\0\0", 8)     = 8
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27246 sendto(9, "\1\1\0\0\24CREATE;game;2;Nastya", 25, 0, NULL, 0) = 25
27246 recvfrom(9, "\1\1\0\0.SUCCESS;Game created. Waiti"..., 8192, 0, NULL, NULL) = 51
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 write(1, "\r                                "..., 61) = 61
27247 write(1, "> ", 2)                  = 2
27246 read(6,  <unfinished ...>
27247 write(1, "\r                                "..., 96 <unfinished ...>
27246 <... read resumed>"\1\0\0\0\0\0\0\0", 8) = 8
27247 <... write resumed>)               = 96
27247 write(1, "> ", 2)                  = 2
27246 recvfrom(9, "\1\1\0\0\"NOTIFY;Player 'Dasha' joine"..., 8192, 0, NULL, NULL) = 76
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                "..., 61) = 61
27247 write(1, "> ", 2)                  = 2
27247 write(1, "\r                                "..., 87) = 87
27247 write(1, "> ", 2)                  = 2
27247 write(1, "\r                                "..., 61) = 61
27247 write(1, "> ", 2)                  = 2
27247 write(1, "\r                                "..., 85) = 85
27247 write(1, "> ", 2)                  = 2
27244 <... read resumed>"guess game ELEPHANT\n", 1024) = 20
27244 write(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27244 write(1, "> ", 2)                  = 2
27244 read(0,  <unfinished ...>
27246 read(6, "\1\0\0\0\0\0\0\0", 8)     = 8
27246 sendto(9, "\1\1\0\0\32GUESS;game;Nastya;ELEPHANT", 31, 0, NULL, 0) = 31
27246 recvfrom(9, "\1\1\0\0'NOTIFY;'Nastya' guess 'ELEP"..., 8192, 0, NULL, NULL) = 44
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                "..., 61) = 61
27247 write(1, "> ", 2)                  = 2
```

```
27247 write(1, "\r                                    "..., 92) = 92
27247 write(1, "> ", 2)                     = 2
27246 recvfrom(9, "\1\1\0\0&NOTIFY;'Dasha' guess 'CHAMP"..., 8192, 0,
    NULL, NULL) = 43
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                    "..., 61) = 61
27247 write(1, "> ", 2)                     = 2
27247 write(1, "\r                                    "..., 91) = 91
27247 write(1, "> ", 2)                     = 2
27246 recvfrom(9, "\1\1\0\0,NOTIFY;Player 'Dasha' has l"..., 8192, 0,
    NULL, NULL) = 49
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                    "..., 61) = 61
27247 write(1, "> ", 2)                     = 2
27247 write(1, "\r                                    "..., 97) = 97
27247 write(1, "> ", 2)                     = 2
27244 <... read resumed>"guess game DEADLOCK\n", 1024) = 20
27244 write(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27244 write(1, "> ", 2)                     = 2
27244 read(0,  <unfinished ...>
27246 read(6, "\1\0\0\0\0\0\0\0", 8)     = 8
27246 sendto(9, "\1\1\0\0\32GUESS;game;Nastya;DEADLOCK", 31, 0, NULL,
    0) = 31
27246 recvfrom(9, "\1\1\0\0*NOTIFY;Player 'Nastya' WON!"..., 8192, 0,
    NULL, NULL) = 47
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                    "..., 61) = 61
27247 write(1, "> ", 2)                     = 2
27247 write(1, "\r                                    "..., 95) = 95
27247 write(1, "> ", 2)                     = 2
27246 recvfrom(9, "\1\1\0\0\vSERVER_STOP", 8192, 0, NULL, NULL) = 16
27246 write(8, "\1\0\0\0\0\0\0\0", 8)    = 8
27246 recvfrom(9, "", 8192, 0, NULL, NULL) = 0
27246 write(6, "\1\0\0\0\0\0\0\0", 8)    = 8
27246 read(6, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 read(8, "\1\0\0\0\0\0\0\0", 8)     = 8
27247 write(1, "\r                                    "..., 61) = 61
27247 write(1, "> ", 2)                     = 2
27247 write(1, "\r                                    "..., 88) = 88
27247 exit_group(0 <unfinished ...>
27244 <... read resumed> <unfinished ...>) = ?
27247 <... exit_group resumed>)             = ?
27247 +++ exited with 0 +++
27246 +++ exited with 0 +++
27245 +++ exited with 0 +++
27244 +++ exited with 0 +++
```